

Unit 2 Notes: Back End Development

Unit 2 Notes: Back End Development

1. Course Orientation

Unit 1 built the front end: a browser that sends a request and renders whatever comes back. Unit 2 builds the other half of that conversation, the server answering it. The unit has seven blocks that build on each other in order:

- **Python Core:** the language features Django leans on constantly, flexible function arguments, exception handling, and object oriented programming.
- **Networking and Files:** what a socket actually is underneath an HTTP request, and how Python reads and writes files on disk.
- **Django MVT:** how a Django project is organised, and how one request travels from a URL to a database row to rendered HTML.
- **Auth and Sessions:** Django's built in login system, and what a session and a cookie actually are under the hood.
- **Database Connectivity:** moving from Django's default SQLite file to a real PostgreSQL server, and how the ORM turns Python into SQL.
- **APIs with Django REST Framework:** returning JSON instead of HTML, so any client, not just a browser, can use the application.
- **Machine Learning Model Integration:** serving predictions from a trained model through a protected API endpoint.

2. Python Core

Django is a Python framework, and it uses a specific slice of the language everywhere: flexible function signatures, disciplined error handling, and classes built around inheritance. This block covers that slice, not Python from scratch.

2.1 `*args` and `**kwargs`

A Python function can accept more arguments than the ones it names explicitly, by collecting the extra ones into a tuple or a dictionary:

```
def order_pizza(size, *toppings, **extras):
    ...

order_pizza("L", "olive", "corn", cheese=True)
```

Parameter	Role	Value in the call above
<code>size</code>	A normal positional parameter	<code>"L"</code>
<code>*toppings</code>	Extra positional arguments, collected into a tuple	<code>("olive", "corn")</code>
<code>**extras</code>	Extra keyword arguments, collected into a dict	<code>{"cheese": True}</code>

`*` collects any positional arguments beyond the named ones into a tuple, and `**` collects any keyword arguments beyond the named ones into a dictionary. This is exactly the mechanism Django itself uses to accept a flexible, growing set of options in its own functions and class methods, so recognising the pattern here makes Django's own signatures far less mysterious later.

The same two symbols also work in reverse, at the call site rather than the definition, to unpack a collection back out into separate arguments: `order_pizza("L", *["olive", "corn"], **{"cheese": True})` sends exactly the same call as writing the arguments out by hand. This is how Django forwards a dictionary of form data or URL keyword arguments straight into a function without listing every key by name.

2.2 Exception handling flow

Python handles errors with a four part block, and each part runs at a different, well defined moment:

Block	Runs when
<code>try</code>	Always. This is the risky code being attempted.
<code>except</code>	Only if an error matched inside <code>try</code> . This is where it is handled.
<code>else</code>	Only if <code>try</code> finished with no error at all.
<code>finally</code>	Always, whether or not an error happened.

```
try:
    risky_operation()
except ValueError as e:
    handle_it(e)
else:
    only_if_no_error()
finally:
    always_runs()
```

Catch a specific error type, for example `except ValueError as e:`, and never a bare `except:`. A bare `except` catches everything, including mistakes you did not anticipate and genuinely want to see, such as a typo turning into a `NameError`, and it silently hides them.

A single `try` can carry several `except` blocks stacked in order, one per error type it needs to handle differently, and Python checks them top to bottom, running only the first one that matches. Django uses this same pattern internally: a view that cannot find a database row typically raises a specific exception, `Http404`, which Django's own error handling then catches further up and turns into a proper 404 response, rather than letting the request crash.

2.3 Classes: a blueprint and its instances

A class is a blueprint, defined once. Each time it is called, it produces an independent object, an instance, with its own copy of whatever state the blueprint describes:

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says woof"
```

`Dog(name="Rex")`, `Dog(name="Milo")` and `Dog(name="Nala")` are three separate objects built from the one `Dog` blueprint, each with its own independent `name`. `__init__` is the constructor, it runs automatically the moment an object is created and is where that object's own starting state is set up. `self` inside a method always refers to the specific instance the method was called on, which is how `self.name` can be different for Rex, Milo and Nala even though they all share the exact same `bark` method, and it is always the first parameter of every ordinary method, even though it is never passed explicitly when the method is called.

Every attribute set with `self.something` in `__init__`, such as `self.name`, belongs to that one instance and is separate for every object. This is different from an attribute written directly inside the class body, outside any method, which is shared across every instance of that class unless a specific object later overrides it.

2.4 Inheritance

A class can be defined in terms of another class, inheriting everything the parent already has and adding or changing only what is different:

```
class Animal:
    def speak(self):
        ...

class Dog(Animal):
    def speak(self):
        return "Woof"

class Cat(Animal):
```

```
def speak(self):  
    return "Meow"
```

`Dog` and `Cat` are both `Animal`s, and both automatically get everything `Animal` defines, without repeating that code. Here, both children also override `speak`, method overriding, meaning a child redefines a method it inherited in order to change just that one piece of behaviour, while everything else stays shared.

When an override still needs the parent's original behaviour as part of its own, `super()` calls that parent version explicitly, for example `super().speak()` inside `Dog.speak`, rather than losing access to it entirely. This matters directly in Django, where a custom class very often overrides one method of a much larger built in class and calls `super()` to still run the framework's own original logic alongside the custom addition.

2.5 Magic (dunder) methods

A magic method, or dunder method, named with leading and trailing double underscores, is a method Python calls automatically in response to a built in operation, rather than one you call directly by name.

Method	Powers	Called as
<code>__init__</code>	Runs when an object is created	<code>obj = Dog()</code>
<code>__str__</code>	What <code>print()</code> shows for the object	<code>print(obj)</code>
<code>__len__</code>	Powers the <code>len()</code> built in	<code>len(obj)</code>
<code>__eq__</code>	Defines what <code>==</code> means for two objects	<code>a == b</code>
<code>__repr__</code>	The developer facing representation	<code>repr(obj)</code>

Without these defined, a custom object prints as a fairly meaningless memory address and cannot be compared with `==` in any way that reflects its actual data. Defining `__str__` and `__eq__` is what makes a custom class feel like it behaves as naturally as Python's own built in types.

`__str__` and `__repr__` are easy to mix up: `__str__` is the readable version meant for a person, what `print()` shows, while `__repr__` is the unambiguous, developer facing version meant for debugging, ideally precise enough to recreate the object. If only one is defined, Python falls back to `__repr__` wherever `__str__` would otherwise be used, which is why `__repr__` is generally worth defining even when `__str__` is also present.

2.6 The four pillars of OOP

Everything in sections 2.3 to 2.5 fits under four named ideas, the four pillars of object oriented programming:

Pillar	Meaning	Where it appears above
Encapsulation	Data and the methods that act on it are bundled together in one class, with internal details kept out of reach	<code>name</code> and <code>bark()</code> live together inside <code>Dog</code>
Abstraction	The caller works through a simple interface without needing to know what happens inside	Calling <code>bark()</code> without knowing how the sentence is built
Inheritance	A child class reuses and extends a parent	<code>Dog(Animal)</code> and <code>Cat(Animal)</code>
Polymorphism	The same method call behaves differently depending on the object it is called on	<code>speak()</code> returns "Woof" or "Meow"

Python has no truly private attributes, so encapsulation is enforced by convention: a single leading underscore, `self._balance`, signals "internal, do not touch from outside", while a double leading underscore, `self.__balance`, makes Python rename the attribute internally, called name mangling, so it is harder to reach by accident.

Polymorphism is what lets code treat different objects in exactly the same way:

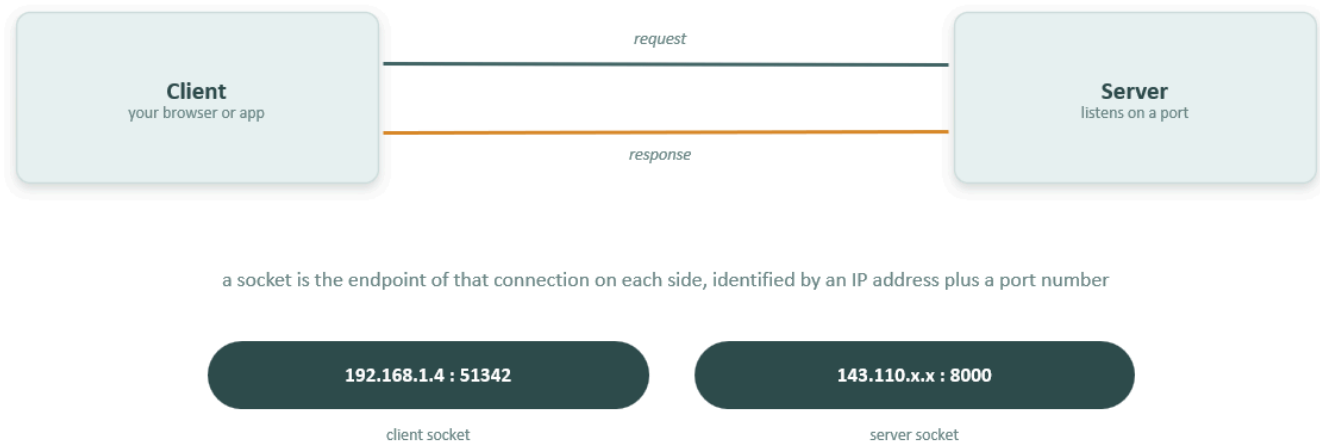
```
for pet in [Dog(), Cat()]:
    print(pet.speak())    # Woof, then Meow
```

The loop never checks which class each object belongs to, it just calls `speak()` and each object answers in its own way. Django relies on this constantly: it calls the same methods on every model and every view, and each one behaves according to its own class.

3. Networking and Files

3.1 Recap: client, server, and what a socket needs

Unit 1 covered the request and response at the level of a browser and a URL. Underneath that exchange, both sides communicate through a socket: the endpoint of a connection on each side, identified by an IP address plus a port number.



Every request travels from a specific client socket to a specific server socket, and the response travels back along the same pairing. A port is simply which of many possible doors on that machine the connection is using, since one server machine can run several services at once, each listening on its own port. Port 8000 in the diagram is Django's own default development port, `python manage.py runserver` listens there unless told otherwise, and port 80 is the default for ordinary, unencrypted HTTP traffic in production.

3.2 Opening a socket in Python

Python's `socket` module gives direct access to this layer, the exact layer that HTTP, and Django underneath it, are built on top of:

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("example.com", 80))
```

Call	Role
<code>socket()</code>	Create an endpoint
<code>connect()</code>	Reach out to a server
<code>send()</code> / <code>recv()</code>	Exchange bytes
<code>close()</code>	Release the connection

HTTP is, underneath all of Django's abstraction, just a specific text protocol spoken over exactly this kind of socket. A request is a block of text sent with `send()`, a response is a block of text read back with `recv()`, and everything Django, or any web framework, does on top of this is convenience built over these four calls. `AF_INET` says the address is a normal IP address, and `SOCK_STREAM` says the connection behaves as TCP, a reliable, ordered stream of bytes, which is the specific guarantee HTTP is built to assume.

3.3 HTTP methods, at a glance

An HTTP method states the intent of a request, what the client is trying to do to a resource, not just which resource it is asking about:

Method	Intent
GET	Read a resource
POST	Create a resource
PUT	Replace a resource
DELETE	Remove a resource

```
GET /api/orders/42 HTTP/1.1
Host: example.com
Authorization: Bearer <token>
```

```
200 OK -> a JSON body with the order details
```

This maps directly onto the request anatomy from Unit 1, method, path, headers, and a response with a status code and a body. Django's views, covered in section 4, are essentially functions written to answer exactly this shape of request, one method and one path at a time, and a single view function commonly checks `request.method` to decide whether it is handling a `GET`, showing a form or a page, or a `POST`, acting on submitted data. `GET` and `PUT` are also expected to be idempotent, repeating them has the same effect as doing it once, which is part of why forms that create something, an order or a comment, are sent as `POST` rather than `GET`.

3.4 Reading and writing files

Python opens a file in one of several modes, and the mode chosen decides both what is allowed and what happens to any existing content:

Mode	Behaviour
r	Read. The file must already exist.
w	Write. Overwrites the file completely.
a	Append. Adds to the end of the file.
r+	Read and write, no truncation of existing content.

```
with open("data.txt", "r") as f:
    text = f.read()
```

The `with` block is the standard pattern for opening a file: the file closes itself automatically the moment the block ends, even if an error happens partway through, which avoids the class of bugs where a file is accidentally left open. Adding `b` to any of these modes, for example `rb` or `wb`, switches to binary

mode, reading or writing raw bytes instead of decoded text, which matters for anything that is not plain text, such as an image or a PDF.

3.5 The server side of a socket

Section 3.2 opened a socket as a client. A server needs three extra calls before it can talk: claim an address, start listening, and accept a client:

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 9000))
server.listen(5)

conn, addr = server.accept()
data = conn.recv(1024)
conn.send(b"Echo: " + data)
conn.close()
```

Call	Role
<code>bind()</code>	Attach the socket to a specific IP and port
<code>listen()</code>	Start waiting for incoming connections, queueing up to 5 here
<code>accept()</code>	Wait for the next client, then return a new socket for it plus its address

`accept()` returns a brand new socket, `conn`, used only for that one client, while the original `server` socket keeps listening for others. `python manage.py runserver` is, at its core, this same loop: accept a connection, read the request text, hand it to Django, and send the response back. The `b"..."` matters too: sockets carry raw bytes, not text, which is why strings are encoded to bytes before sending and decoded back to text after receiving.

3.6 JSON files

JSON is the text format every API in this unit speaks, and Python's `json` module converts between JSON text and ordinary Python dictionaries and lists:

```
import json

order = {"id": 42, "items": ["samosa", "chai"], "paid": True}

text = json.dumps(order)          # dict to JSON string
back = json.loads(text)           # JSON string to dict

with open("order.json", "w") as f:
    json.dump(order, f)           # dict to file
```

```
with open("order.json") as f:
    data = json.load(f)           # file to dict
```

The `s` in `dumps` and `loads` stands for string, without it the same functions read and write files directly. JSON maps almost one to one onto Python: a JSON object becomes a dict, an array becomes a list, and `true`, `false` and `null` become `True`, `False` and `None`. This is exactly the conversion DRF's serializers perform automatically in section 7.

4. Django MVT

4.1 The Model, View, Template cycle

Django organises an application around three kinds of file, each with one clear job, and the pattern is called MVT:

Part	File	Job
Model	<code>models.py</code>	The data
View	<code>views.py</code>	The logic
Template	<code>templates/</code>	The HTML

`urls.py` routes an incoming request to a view, the view asks a model for data, and the view fills that data into a template to produce the final HTML response. Keeping these three concerns in three different kinds of file, rather than mixed together, is the whole reason MVT scales to a large application: a designer can edit a template without touching Python, and the data layer can change without rewriting every page that displays it.

MVT is Django's own name for a pattern most other frameworks call MVC, model, view, controller. The mapping is not one to one by name: what Django calls a view is closer to what MVC calls the controller, the logic that decides what happens, while the actual rendering step, what MVC calls the view, is Django's template.

4.2 Project vs app

A Django project and a Django app are not the same thing, and confusing them is a common early mistake:

myproject/	myapp/
<code>settings.py</code>	<code>models.py</code>
<code>urls.py</code>	<code>views.py</code>
<code>wsgi.py</code>	<code>urls.py</code>
	<code>templates/</code>

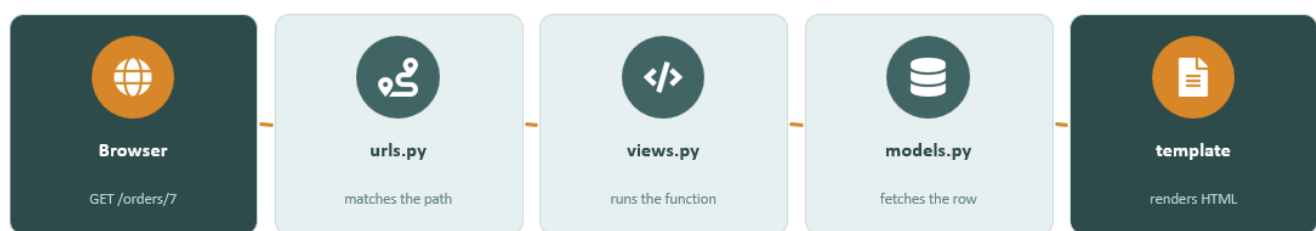
The project is one per site: it holds the global settings and the root URL table that every request first arrives at. An app is one per feature, for example orders, accounts, or payments, and a single project

can, and usually does, hold many apps. This split is what makes an app reusable across different projects, since an app does not hardcode anything about the specific project it happens to be plugged into.

An app only actually takes effect once it is listed in `INSTALLED_APPS` inside the project's `settings.py`. Creating the app's folder with `startapp` is not enough on its own, Django will not pick up its models or templates until that registration step is done, which is a common source of a model or template silently appearing not to exist.

4.3 Anatomy of one request

Putting MVT and the project/app split together, a single request makes a fixed round trip through five steps:



the response makes the same trip back, template → view → browser

The response then makes the same trip back in reverse, template to view to browser. Every Django request, however complex the page, is fundamentally this same five step trip: a URL is matched, a view function runs, that function may ask a model for data, and a template turns the result into HTML to send back. A view does not have to render a template at all, it can just as easily return a redirect to a different URL, or return data directly as JSON instead of HTML, which is the same basic pattern an API endpoint built with Django REST Framework, covered in section 7, is built on.

4.4 From a model to a database table

A model is a Python class, but Django turns it into a real SQL table through a two step, version controlled process:

```
class Order(models.Model):  
    ...
```

Step	Command	What happens
1	Define the model	Write the class in <code>models.py</code>
2	<code>makemigrations</code>	Write the change as a migration file
3	<code>migrate</code>	Apply that file to the database
Result	<code>orders_order</code>	The real table now exists

Migrations are version control for the database schema: every change to a model is captured as its own file, in order, and that file is committed and shared with the team through Git exactly like any other source file, covered in Unit 1. This is what lets a whole team keep their local databases in sync with the model definitions as those definitions evolve. `makemigrations` only writes the migration file, it does not touch the database, `migrate` is the separate step that actually applies any pending migration files, which is why a change to `models.py` that is never followed by both commands never actually reaches the real table.

4.5 Templating basics

A Django template is HTML with two small additions: a way to print a value, and a way to add simple logic, without ever running arbitrary Python inside the page itself.

```
<h1>Hi {{ user.name }}</h1>

{% if orders %}
    <p>{{ orders|length }} orders</p>
{% endif %}
```

Syntax	Purpose
<code>{{ variable }}</code>	Prints a value that the view passed into the template's context
<code>{% tag %}</code>	Logic: <code>if</code> , <code>for</code> , <code>block</code> , and so on

The deliberate restriction here matters: a template can display and lightly branch on data, but it can never execute raw Python, which keeps the separation between logic (in `views.py`) and presentation (in the template) enforced by the framework itself, rather than left as a convention someone can break.

Two further pieces come up in almost every real template. A filter, the `|length` after `orders` above, transforms a value inline, other common ones include `|date` and `|default`. Template inheritance lets one base template define the shared page layout once, with `{% block content %}` marking the parts a specific page is allowed to fill in, and every other template then opens with `{% extends "base.html" %}` instead of repeating the surrounding HTML on every single page.

4.6 Model fields and relationships

Section 4.4 left the `Order` model's body empty. Filled in, it looks like this:

```
from django.db import models
from django.contrib.auth.models import User

class Order(models.Model):
    customer = models.ForeignKey(User, on_delete=models.CASCADE)
    item = models.CharField(max_length=100)
    quantity = models.IntegerField(default=1)
```

```
price = models.DecimalField(max_digits=8, decimal_places=2)
paid = models.BooleanField(default=False)
created_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return f"{self.item} x {self.quantity}"
```

Each field is a class attribute, section 2.3, because it describes the whole table rather than one row, and each one becomes a column. Django adds an `id` primary key column automatically, so it never needs to be declared.

Field	Stores
<code>CharField</code>	Short text, <code>max_length</code> is required
<code>TextField</code>	Long text with no length limit
<code>IntegerField</code> / <code>FloatField</code>	Whole numbers / decimal numbers
<code>DecimalField</code>	Exact decimals, the right choice for money
<code>BooleanField</code>	<code>True</code> or <code>False</code>
<code>DateTimeField</code>	A date and time, <code>auto_now_add</code> sets it once on creation

Models are connected to each other with three relationship fields:

Field	Relationship	Example
<code>ForeignKey</code>	Many to one	Many orders belong to one user
<code>OneToOneField</code>	One to one	One profile per user
<code>ManyToManyField</code>	Many to many	Students and courses

`on_delete` says what happens to an order when its user is deleted: `CASCADE` deletes the orders too, `PROTECT` blocks deleting the user, and `SET_NULL` keeps the orders with an empty customer. The `__str__` method at the bottom is the dunder method from section 2.5, and it is what labels each order in Django's admin site.

4.7 Writing a view and wiring its URL

A view is a Python function that takes a `request` and returns a response. Most views fetch some data and hand it to a template with `render()`:

```
# orders/views.py
from django.shortcuts import render, get_object_or_404
from .models import Order
```

```
def order_list(request):
    orders = Order.objects.filter(paid=False)
    return render(request, "orders/list.html", {"orders": orders})

def order_detail(request, pk):
    order = get_object_or_404(Order, pk=pk)
    return render(request, "orders/detail.html", {"order": order})
```

`render()` takes the request, a template name, and a context: a dictionary whose keys become the variable names inside the template, so the `orders` key here is exactly what `{% if orders %}` in section 4.5 reads. `get_object_or_404` is the `Http404` pattern from section 2.2 in one line: if the row does not exist, the user sees a proper 404 page instead of a crash. The query itself, `filter(paid=False)`, is covered in section 6.3.

A view does nothing until a URL points at it:

```
# orders/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path("", views.order_list, name="order-list"),
    path("<int:pk>/", views.order_detail, name="order-detail"),
]

# myproject/urls.py
from django.urls import path, include

urlpatterns = [
    path("orders/", include("orders.urls")),
]
```

The project's root table hands everything under `orders/` to the app's own `urls.py` with `include()`, keeping each app's URLs inside that app, section 4.2. The `<int:pk>` part is a path converter: it matches digits, converts them to an integer, and passes the value into the view as a keyword argument, the `**kwargs` idea from section 2.1. So a request for `/orders/7/` calls `order_detail(request, pk=7)`, which is the full trip from the diagram in section 4.3. The `name` on each route lets templates and views refer to a URL by name, for example `{% url 'order-detail' order.id %}`, so the path itself can change in one place only.

A single view often handles both showing a form and acting on it, checking `request.method` from section 3.3: on `GET` it renders the form, on `POST` it saves the data and then returns a `redirect()` rather than a page, so that refreshing the browser afterwards does not submit the form a second time.

5. Auth and Sessions

5.1 `django.contrib.auth`, out of the box

Django ships with a complete authentication system already built, `django.contrib.auth`, so a login flow does not need to be written from scratch:

```
from django.contrib.auth import authenticate, login

user = authenticate(
    request, username=u, password=p
)
if user is not None:
    login(request, user)
```

`authenticate()` checks a username and password against the database and returns the matching user, or `None` if they do not match. `login()` then attaches that user to the current session, covered in section 5.3. The built in `User` model already carries the fields a real login system needs: `username`, a `password` that is always hashed, never stored as plain text, and the three boolean flags `is_active`, `is_staff` and `is_superuser` that control what the account is allowed to do. A new user is created with `User.objects.create_user(...)`, never `User(...)` directly, since `create_user` is what actually runs the password through Django's hashing before saving it. Logging out is the direct counterpart, `logout(request)` clears the session so the next request from that browser is anonymous again.

5.2 Permissions and decorators

Once a login system exists, individual views need to be protected, restricted to logged in users, or further restricted to users holding a specific permission. Django does this with decorators placed directly above a view function, plus two checks usable inside a view or a template:

Tool	Effect	Where
<code>@login_required</code>	Redirects anonymous users to the login page	<code>views.py</code>
<code>@permission_required</code>	Blocks the view unless the user holds that permission	<code>views.py</code>
<code>request.user.is_authenticated</code>	<code>True</code> once a user is logged in	templates and views
<code>request.user.has_perm(...)</code>	Checks one specific permission in code	<code>views.py</code>

A decorator wraps a view function without editing its body, so protecting a view is usually a one line addition directly above its definition, rather than a block of conditional logic written inside every protected view by hand. Individual permissions are normally bundled into a Group, for example "editors"

or "support staff", rather than assigned to each user one by one, so `@permission_required` in practice is usually checking membership in one of those groups.

5.3 What a session actually is

A session is how Django recognises the same browser across multiple separate requests, and understanding it means separating what lives in the browser from what lives on the server:

Browser	Server
cookie: sessionid = 8k3j...f91	--> django_session table: key · data · expiry

Only a session ID travels in the cookie sent to the browser, the real session data itself stays on the server, stored in the `django_session` table. This is why `login()` just works: Django writes one row into that session table, sets one cookie on the response containing nothing but the key to that row, and every later request from that same browser carries the cookie back, letting Django look up the matching row and know which user is making the request, without the password or any sensitive data ever needing to travel back and forth again.

A session row also carries an expiry, `SESSION_COOKIE_AGE` in settings controls how long, by default, a session stays valid, and past that point the cookie no longer matches any live row and the user is treated as logged out again. Sessions are also what makes CSRF protection possible: Django ties a one time token to the session and expects every `POST` form to send it back, which is how it tells a genuine submission from your own page apart from one forged by another site.

6. Database Connectivity

Every model so far has quietly lived in SQLite, the database Django picks by default: the whole database is a single file, `db.sqlite3`, sitting next to `manage.py`. That is ideal for learning, but a real application runs on a database server such as PostgreSQL, and connecting to one is the first step towards an application more than one person can use at a time.

6.1 SQLite vs PostgreSQL

	SQLite	PostgreSQL
Where it lives	One file inside the project	A separate server process, port 5432
Setup	None, Django uses it by default	A running server plus settings
Many users writing at once	Weak, it locks the whole file	Strong, it is built for it
Typical use	Learning, prototypes	Production applications

Because Django only ever talks to the database through its ORM, switching from one to the other is a configuration change, not a rewrite. The models, views and templates from section 4 stay exactly the

same.

6.2 Pointing Django at PostgreSQL

Python needs a driver to talk to PostgreSQL, `psycopg2`, installed with `pip install psycopg2-binary`. Django is then told where the server is through the `DATABASES` setting:

```
# settings.py
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "imapdb",
        "USER": "postgres",
        "PASSWORD": "yourpassword",
        "HOST": "localhost",
        "PORT": "5432",
    }
}
```

`ENGINE` picks the database backend, `NAME` is the database inside the server, and `HOST` and `PORT` are exactly the IP and port pairing from section 3.1. The new database starts empty, so `python manage.py migrate` is run again, and every table, users and sessions included, is rebuilt inside PostgreSQL. In a real project the password is not typed into `settings.py`, which is committed to Git, but read from an environment variable instead.

6.3 Querying through the ORM

Every model gets a manager called `objects`, and queries are written in Python rather than SQL:

```
Order.objects.all()           # every row
Order.objects.get(id=7)       # exactly one row
Order.objects.filter(paid=False) # rows that match
Order.objects.filter(price__gte=50) # price >= 50
```

`filter()` and `all()` return a `QuerySet`, a description of a query rather than the rows themselves. `QuerySets` are lazy: no SQL actually runs until the data is needed, when the `QuerySet` is looped over, counted or displayed in a template, which is why filters can be chained step by step without hitting the database each time. To see the SQL Django writes, print the `QuerySet`'s query in `python manage.py shell`:

```
>>> print(User.objects.filter(is_active=True).query)
SELECT ... FROM "auth_user" WHERE "auth_user"."is_active" = true
```

This works because `print()` calls the object's `__str__`, the dunder method from section 2.5.

6.4 Raw SQL, and why placeholders matter

The ORM covers almost everything, but Django also allows raw SQL when it is genuinely needed:

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute("SELECT username FROM auth_user WHERE id = %s", [user_id])
    row = cursor.fetchone()
```

The value is never glued into the SQL string, it is passed separately and marked with a `%s` placeholder. Gluing user input straight into SQL opens the door to SQL injection, where someone types SQL into a form field and it runs on your database. With placeholders, the driver escapes every value safely, and the ORM does this automatically for every query it writes, which is one of the strongest reasons to prefer it.

7. APIs with Django REST Framework

Everything so far returns HTML, which only a browser can really use. An API returns data instead, usually JSON, so any client can use it: a React front end in Unit 4, a mobile app, another server, or a script. Django REST Framework, DRF, is a package that sits on top of Django and adds exactly three new ideas to build one: serializers, ViewSets and routers.

7.1 From HTML views to a JSON API

	HTML views (section 4)	API views (this section)
Returns	A rendered HTML page	JSON data
Used by	A browser	Any client
Staying logged in	A session cookie	A token sent with every request

A REST API organises its URLs around resources and lets the HTTP method from section 3.3 say what to do: `GET /api/users/` lists users, `POST /api/users/` creates one, `GET /api/users/5/` reads one, `PUT` replaces it and `DELETE` removes it. DRF is installed with `pip install djangorestframework` and registered like any app, by adding `"rest_framework"` to `INSTALLED_APPS`.

7.2 Serializers

A serializer is the translator between a model instance and JSON:

```
from rest_framework import serializers
from django.contrib.auth.models import User

class UserSerializer(serializers.ModelSerializer):
    class Meta:
```

```
model = User
fields = ["id", "username", "email"]
```

It works in both directions. Going out, it turns a `User` object into JSON for a response. Coming in, it validates incoming JSON before anything touches the database, and rejects bad data, a missing field or a badly formed email, with a `400` and a clear error message. A `ModelSerializer` reads the model's own field definitions and builds matching fields automatically, and the `fields` list decides exactly what is exposed, which is why the hashed password from section 5.1 can never leak through this API.

7.3 ViewSets and routers

A `ViewSet` bundles every action for one resource into a single class:

```
from rest_framework import viewsets

class UserViewSet(viewsets.ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
```

These two lines give list, create, retrieve, update and delete, what would otherwise be five separate views. This is inheritance from section 2.4 doing the heavy lifting: all the logic lives in `ModelViewSet`, and the child class only fills in what is different. A router then writes the URLs for it:

```
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register("users", UserViewSet)
urlpatterns += [path("api/", include(router.urls))]
```

One registration creates both `/api/users/` and `/api/users/<id>/`, each wired to the right action. Not every endpoint is plain CRUD on a model, and for custom logic, such as the ML prediction in section 8, a view inherits from `APIView` instead and defines one method per HTTP verb it answers.

7.4 Token authentication

A browser stays logged in with the session cookie from section 5.3, but a mobile app or a script has no browser. With token authentication, each user is issued a long random string, and the client sends it in a header with every request:

```
Authorization: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b
```

Section 3.3 showed the same idea with the word `Bearer`, which many APIs use. DRF's built in token system uses the word `Token` instead. Turning it on takes two settings:

```
INSTALLED_APPS += ["rest_framework.authtoken"]

REST_FRAMEWORK = {
```

```
"DEFAULT_AUTHENTICATION_CLASSES": [  
    "rest_framework.authentication.TokenAuthentication",  
    "rest_framework.authentication.SessionAuthentication",  
],  
}
```

Adding `authtoken` alone only creates the table that stores the tokens.

`DEFAULT_AUTHENTICATION_CLASSES` is what actually makes DRF read the header, and keeping session authentication in the list lets a logged in browser use the same API too. A token for a user is created with `python manage.py drf_create_token <username>`.

7.5 Permissions

Authentication only answers who is asking. Whether that user is allowed through is decided by permission classes, set on the view:

Permission class	Allows
<code>AllowAny</code>	Everyone. This is DRF's default.
<code>IsAuthenticated</code>	Only requests with a valid session or token
<code>IsAdminUser</code>	Only users with <code>is_staff = True</code>
<code>IsAuthenticatedOrReadOnly</code>	Anyone may read, only logged in users may change data

The default matters more than it looks: a view with no `permission_classes` is open to everyone, even with token authentication fully configured, because authentication identifies users but never blocks them. Adding `permission_classes = [IsAuthenticated]` to a view is what turns a missing or wrong token into a `401` with "Authentication credentials were not provided."

8. Machine Learning Model Integration

A trained ML model is only useful once other people and programs can send it inputs and get predictions back. Wrapping it in a DRF endpoint does exactly that: a client sends the input features as JSON and receives the prediction as JSON. The example throughout is the Iris dataset, four flower measurements in, one of three species out.

8.1 Train once, save to disk

Training happens once, in a separate script that is not part of the web application, and the trained model is saved to a file with `joblib`:

```
from sklearn.datasets import load_iris  
from sklearn.ensemble import RandomForestClassifier  
import joblib
```

```
X, y = load_iris(return_X_y=True)
clf = RandomForestClassifier().fit(X, y)
joblib.dump(clf, "iris_model.pkl")
```

Training can take minutes or hours, while a web request should answer in milliseconds, so the server never trains, it only ever uses a finished model. The `.pkl` file is binary, the `wb` and `rb` modes from section 3.4, and it is kept inside the app that serves it, for example

```
mlpredict_iris/ml/iris_model.pkl.
```

8.2 Load once, not per request

```
# mlpredict_iris/ml_model.py
import joblib
from pathlib import Path

MODEL_PATH = Path(__file__).resolve().parent / "ml" / "iris_model.pkl"
model = joblib.load(MODEL_PATH)
```

Python runs a module's top level code only the first time it is imported, so the model is read from disk once, when the server starts, and then shared by every request. Loading it inside the view instead would re-read the file on every single prediction and make the API crawl. `pathlib` builds the path relative to this file, so the model is found no matter which folder the server was started from.

8.3 The prediction endpoint

The input has no database model behind it, just four numbers, so it is validated with a plain `Serializer` rather than a `ModelSerializer`, and the endpoint is an `APIView`, since a prediction is a custom action rather than CRUD:

```
class IrisInputSerializer(serializers.Serializer):
    sepal_length = serializers.FloatField()
    sepal_width = serializers.FloatField()
    petal_length = serializers.FloatField()
    petal_width = serializers.FloatField()

class PredictView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        s = IrisInputSerializer(data=request.data)
        s.is_valid(raise_exception=True)
        d = s.validated_data
        p = model.predict([[d["sepal_length"], d["sepal_width"],
                           d["petal_length"], d["petal_width"]]])[0]
        return Response({"prediction": int(p)})
```

Line	Why it is there
<code>permission_classes = [IsAuthenticated]</code>	Without it, the <code>AllowAny</code> default from section 7.5 lets anyone use the model
<code>is_valid(raise_exception=True)</code>	Bad input is stopped with a <code>400</code> before it reaches the model
<code>[...]</code> , a list inside a list	scikit-learn always expects a batch of rows, even for one prediction
<code>int(p)</code>	The result is a NumPy integer, which cannot be turned into JSON directly

Called with a valid token, the endpoint returns `{"prediction": 0}`, setosa. Called without one, it returns the `401` from section 7.5, which proves the protection actually works.

8.4 Calling it from a browser page

The same endpoint can also be called by a logged in user from a web page, using the `fetch`, `async` and `await` from Unit 1. No token is pasted into the page: the browser sends its session cookie automatically, and the session authentication kept in section 7.4 accepts it. Because a session is involved, the CSRF protection from section 5.3 applies, so the page reads the `csrftoken` cookie and sends it back in an `X-CSRFToken` header, the JavaScript equivalent of `{% csrf_token %}` in a form. This one page ends up using almost every idea in the unit together: a protected view, a template, a session, CSRF, a serializer, permissions, and a model loaded from a file.

9. Quick Recap

- `*args` / `**kwargs`: collect extra positional and keyword arguments.
- `try` / `except` / `finally`: handle errors without crashing the program.
- Classes and inheritance: a blueprint, its instances, and children that override methods.
- Four pillars of OOP: encapsulation, abstraction, inheritance and polymorphism.
- Sockets and HTTP: the request and response exchange underneath every web call, with clients that connect and servers that bind, listen and accept.
- MVT: model holds data, view holds logic, template holds HTML.
- Views and URLs: `urls.py` maps a path to a view, and `render()` fills a template with a context dictionary.
- Migrations: the version controlled path from a model to a real database table.
- Auth and sessions: `login()` sets one cookie, the real data stays server side, a protected view now guards real users, with no third party package required.
- Database connectivity: moving from SQLite to PostgreSQL is a `DATABASES` change plus a driver, QuerySets are lazy, and raw SQL always uses placeholders.

- DRF: a serializer translates between models and JSON, a ViewSet gives full CRUD in one class, and a router writes the URLs.
- API auth: a token in the `Authorization` header identifies the user, and `IsAuthenticated` is needed because the default is `AllowAny`.
- ML integration: train once, load once at import, validate input with a serializer, and return the prediction as JSON.